

PROJECT REVIEW BRIEF

Smoker

Docker-based CPAN dependency testing framework

To: Doug Bell

From: Raymond Mannarelli

Date: 30 August 2026

Purpose: Technical review and comments

Smoker is a Docker-based framework for testing CPAN modules under controlled Perl and dependency configurations. It grew out of the Smoker work discussed during the Chicago Perl Mongers period and has since been substantially rebuilt and extended.

The project now has an operational implementation, automated regression and recovery tests, independent batch validation, reconciled technical documentation, and evidence from a completed 61,035-case multi-version test exercise followed by an extensive forensic audit.

1. Purpose and scope

Smoker tests CPAN modules while deliberately controlling selected dependency versions.

It supports three principal modes:

- **baseline** — test the target module without deliberately varying dependencies;
- **vary-one** — test with one specified dependency version;
- **vary-two** — test with two specified dependency versions.

Each planned case executes independently in a Docker container and preserves durable result evidence.

The central execution invariant is:

One unique plan row -> one run directory -> one rc.code -> one summary row

The framework is intended not only to record PASS or FAIL, but to preserve enough evidence to investigate why a particular result occurred.

2. Current implementation

The principal components are:

- `Run_Smoker.pl` — launcher and top-level execution control;
- `bin/run_plan_csv.pl` — scheduler command-line interface;
- `lib/Smoker/Scheduler.pm` — scheduler-parent run-directory reservation, worker dispatch, and recovery;
- `lib/Smoker/Runner.pm` — per-test execution coordination;
- `lib/Smoker/Inner.pm` — container-side dependency and target operations;
- `lib/Smoker/Docker.pm` — Docker execution and process handling;
- `lib/Smoker/Result.pm` — result classification and finalization;
- `bin/validate_smoker_batch.pl` — independent batch validator.

The implementation includes explicit handling for worker failure, scheduler interruption, incomplete result state, duplicate or damaged summary state, and evidenced framework failures.

Summary maintenance uses locking, a run-ID membership index, per-run transaction state, an optimized append path for ordinary first-time results, and update-or-insert reconciliation when recovery or retry requires it.

Framework failures use `rc=125` and require explicit `framework_error` classification and causal evidence. They are kept distinct from failures produced by the module or requested dependency environment.

3. Validation status

The largest completed exercise contains **61,035 test cases across five Perl versions**.

The historical results were:

- **18,398 PASS**
- **31,077 FAIL**
- **11,560 UNAVAILABLE**

Those numbers require an important qualification.

The unexpectedly high historical FAIL count was not accepted as 31,077 demonstrated current module or dependency failures. The original results were preserved unchanged and followed by targeted confirmation testing and a forensic recalibration of the failure population.

The subsequent work examined target-release availability, dependency-install failures, configuration failures, missing prerequisites, target-download failures, dependency versions that did not remain installed, timeout behavior, and local-MiniCPAN effects.

All 61,035 historical rows are now accounted for either by their original non-failure classification or by an assigned causal recalibration category.

Some conclusions come from direct reruns of exact historical cases. Others use repeated pair-level confirmation across the five Perl bases. The large historical dependency-did-not-remain population is explicitly treated as sample-supported rather than as though every historical row had been individually rerun.

The original summary.csv was not rewritten to incorporate these later conclusions. Historical observations remain historical observations; later evidence refines their interpretation.

The detailed results and limitations are documented in:

`Smoker_61035_Final_Forensic_Audit_Report.md`

4. Execution and evidence integrity

Smoker is designed so that admitted test work remains accountable even when execution does not complete normally.

Among the mechanisms tested are:

- permanent run-directory reservation before worker execution;
- exactly one current summary row per run;
- recovery of missing or malformed result artifacts;
- duplicate-summary reconciliation;
- worker signal and abnormal-exit recovery;
- scheduler interruption handling;
- explicit framework-failure evidence;
- preservation of already complete PASS, FAIL, and UNAVAILABLE results;
- independent validation of persisted result evidence.

The validator does not rely on the producer's own assertion that a batch is valid. It independently checks execution accounting, required artifacts, result classification, and summary/run-directory consistency.

Deliberately damaged test fixtures are used to verify that the validator rejects inconsistent or incomplete evidence.

5. Suggested review path

A relatively short review can begin with three documents:

1. **Overview** — describes what Smoker does and its operating model.
2. **Software Design Description** — describes the architecture, result lifecycle, recovery mechanisms, and design boundaries.
3. **61,035-Run Failure-Recalibration Forensic Audit** — describes what was learned from the large validation exercise and the subsequent confirmation work.

For a hands-on check, use the validated 20-row example in the repository README: `plan config/plans/20_tests.csv`; compact evidence examples/test_results/20_tests_20260902_184920/. Then run the automated suite and independent validator.

The source repository contains the implementation and version history for deeper examination if desired.

6. Known boundaries

Smoker depends operationally on Docker, storage capacity, appropriate Perl base images, and access to the required CPAN or MiniCPAN material.

Historical CPAN releases and prerequisites are not always obtainable. Local-mirror absence is not necessarily equivalent to true CPAN unavailability, which is one reason later confirmation was required during the forensic audit.

Handled worker and scheduler failures have explicit recovery mechanisms, but host failure, storage failure, or destruction of the scheduler before its recovery path can execute remain outside the guarantees of normal in-process recovery.

Most development and testing has been performed on one system. Independent execution on another system would therefore be useful additional evidence.

7. What I would value from review

I would particularly value comments on:

- whether the underlying testing problem is still useful;
- whether the execution and evidence model is technically credible;
- whether there are design weaknesses I have overlooked;
- whether the forensic interpretation of the large test run is appropriately cautious;

- whether there are changes that would materially improve the project.

I am not asking you to take over the project, and I am not submitting it for CPAN inclusion. My immediate purpose is to show you what the old Smoker work developed into and get your reaction to it.

8. Development context

The project originated in the earlier Chicago Perl Mongers Smoker work.

More recently, AI assistance has been used extensively for implementation, refactoring, debugging, test development, documentation, and analysis. The testing objectives and empirical approach derive from the original project, while AI assistance made it practical to carry the implementation, validation, and forensic analysis substantially further.

The repository history, source, tests, and retained result evidence are intended to make the work inspectable rather than requiring the conclusions to be accepted on description alone.

9. Review materials

The review material includes:

- source code and repository history;
- reconciled Smoker Overview;
- reconciled Software Design Description;
- documentation map and operating material;
- automated regression, recovery, and validator tests;
- compact retained evidence from a completed 20-row validation example;
- the 61,035-case forensic audit report and its supporting evidence.

Smoker should be regarded as a working technical project being presented for review, rather than as a finished supported product or a request for someone else to assume its maintenance.